Algorithms and data structures

DB4002
20 April 2026

HÖGSKOLAN I HALMSTAD

Assignment Package 3 – Lab2

General instructions: How and when to submit and what to submit are explained on the submission page on Blackboard. This document describes the assignments.

This information is available as part of the course description available in Blackboard:

- All assignment packages are divided into two parts called **Part 1** and **Part 2**. Part 1 is done to get a pass on the exam, while Part 2 is done to get bonus points for a higher grade (4, 5) on the second part of the exam (see the information below about the exam structure). That's it, Part 1 must be done to get a pass, while Part 2 is only done to collect bonus points for a higher grade on the exam.
- The total number of bonus points you can get by solving the tasks on Part 2 for each assignment package is **10**. That' it, you can collect half of the points for a higher grade on the exam by solving these Part 2 tasks. The bonus points you get for the exam are equal to the sum of the bonus points you have received from the solutions to the Part 2 tasks.

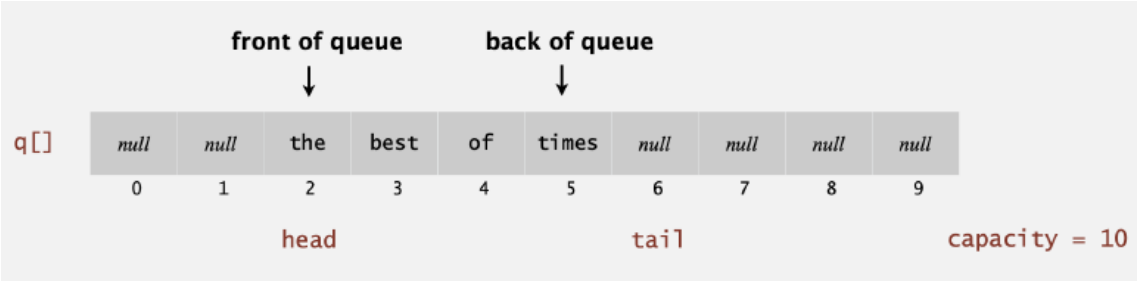
Algorithms and data structures

DB4002
20 April 2026

HÖGSKOLAN I HALMSTAD

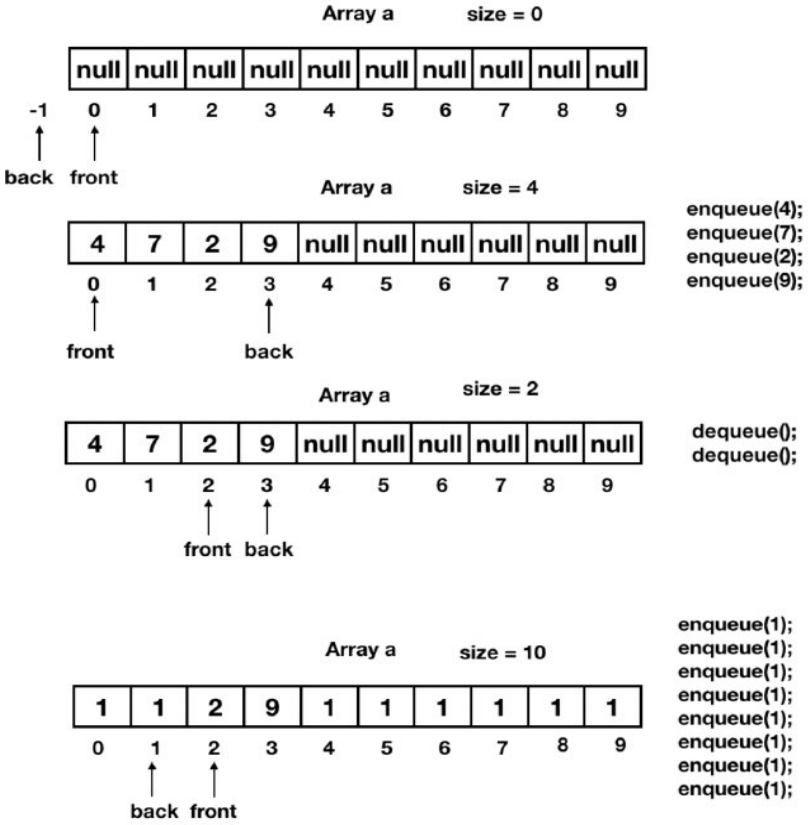
Assignments - part 1

1. In this task, you must implement a class **DataBuffer** that represents a data buffer where you can store objects before they are processed. This buffer has a certain size, **bufferSize**, which must be adjustable (as in **Xarray** and in priority queue implementations, but in this case using a method that the client application may choose to use). A buffer behaves like a queue: **First In First Out**, and can be implemented using an idea commonly called a *circular array*. The following Figure from the course book illustrates this idea:



The field that contains the elements in the buffer is called **q** in the Figure. To implement **First In First Out** efficiently, one must keep track of the first element and the last element in the queue. The idea is that, instead of continuously moving elements to the beginning of the array to utilize the entire array, one can wrap around the positions when the last element is at the last index while there are empty positions at the beginning.

The idea of how this works is illustrated with the following example (the explanation comes after the Figure).



Algorithms and data structures

DB4002
20 April 2026

We have a buffer with a bufferSize (capacity) of 10. The buffer is represented by an array a of length 10. We have two variables, back and front, which indicate the **index** of the first element in the **queue** (front) and the last element (back), respectively. Initially, the buffer contains no elements, **but then we add 4 elements: 4, 7, 2, 9.** We update back each time we add an element. Then we remove 2 elements and update front each time we remove an element. **Then we add eight elements (all ones)** and update back for each insertion. Note that when we need to insert at index 10, we must wrap around to index 0 (we circulate in the array). This is also needed for front and is achieved using modulo arithmetic with the array size.

a) (3 points): The first task is to implement the following API using an array as illustrated on the previous page. If the buffer is full and an attempt is made to add an element, a runtime error should be generated. It should also not be possible to set a buffer size (**capacity**) smaller than 1. Create the **DataBuffer.java** file.

```
public class DataBuffer<T> implements Iterable<T>
-------------------------------------------------------------
// Creates a buffer with the specified capacity:
public DataBuffer(int bufferSize)

// Add t to the last element of the buffer
public void enqueue(T t)

// remove the first element of the buffer and return the value
public T dequeue()

// Size the second buffer to the new specified size.
// If the new size is less than the current capacity
// then any value that are last in the buffer (queue)
// that do not fit are discarded.
public void changeBufferSize(int newBufferSize)

// Is the buffer full?
public boolean isFull()

// Is the buffer empty?
public boolean isEmpty()

// Number of elements in the buffer
public int size()

// Length of the field
public int bufferSize()

// An iterator for the elements in the buffer
public Iterator<T> iterator()
```

b) (1 point): Now we will extend the implementation of a priority queue using a complete binary max heap as done in the lecture. The only new requirement is that you should add an additional constructor as follows (Create the **MaxPQ.java** file):

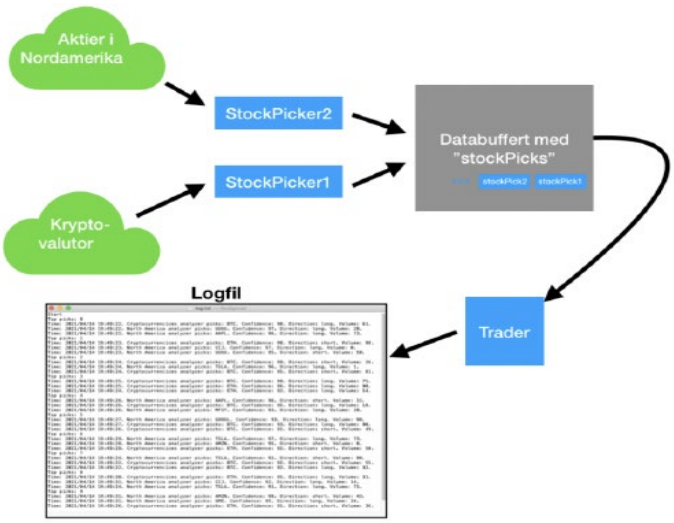
public MaxPQ(T[] a)
The constructor takes an array a as argument and creates a binary max heap represented as an array using O(N) comparisons, where N is the length of the array a. This is done in the following two steps:
i. Place the elements of array a into positions 1 through N in a new array of length N+1.
ii. Start at index N/2 in the new array and iterate through the array, index by index, until you reach position 1. In each iteration, use the sink function to let the element (at the current index) move down in the array (by using the sink method).

Done! The new array is now a heap.

c) (2 points): Now DataBuffer is used in an example to simulate automated trading with financial assets. This is done using the three classes **StockPick**, **StockPicker**, and **Trader**, which are provided in the attached files *StockPick.java, StockPicker.java, and Trader.java*.

We want the simulation to function as described below. A **StockPicker**, as defined here, is an algorithm that selects **stocks** for investment. This occurs in some market. In the example below, we have one **StockPicker** for **U.S. stocks** and one for cryptocurrencies. These two **StockPickers** generate around 10 investment tips per second (the term "investment" is somewhat misleading since this refers to high-frequency trading). An investment tip is an object of type *StockPick* that contains information about the asset, the timestamp, the position to take (long, short), and the volume. Additionally, each StockPick has a confidence value between 0 and 100 (where 100 is best) indicating how good the investment is.

Because the investment tips (**StockPicks**) quickly overflow, the trader chooses to place these StockPicks in a data buffer (i.e., an object of type DataBuffer). The buffer is then emptied once per second, and only the k (e.g., 3) best StockPicks are saved in a log file called log.txt. In this simulation, execution occurs in multiple threads. Instead of having a single execution thread for the program, there is one thread for each StockPicker and one for the Trader. These *three threads* read from and write to the same buffer.
The entire process is summarized in the illustration:



Task: As Trader is currently implemented, nothing interesting happens when we run the program using:

java Trader

Go into the **Trader class** and **add code** to the run method so that every second the nrPicks best investment tips are retrieved from the data buffer stockPicks and written to the log file **log.txt**. Use the priority queue **MaxPQ** to retrieve the nrPicks best investment tips.

In a program with multiple threads, the run method is like the main method in a single-threaded program. There may be multiple run methods started to execute simultaneously. To ensure that multiple threads do not simultaneously add and remove elements from the buffer, you can, for example, add the synchronized modifier to the methods in DataBuffer (but not to the constructor):

```
public synchronized void enqueue(T element)
public synchronized T dequeue()
```

To add data to log.txt, you can, for example, create an object

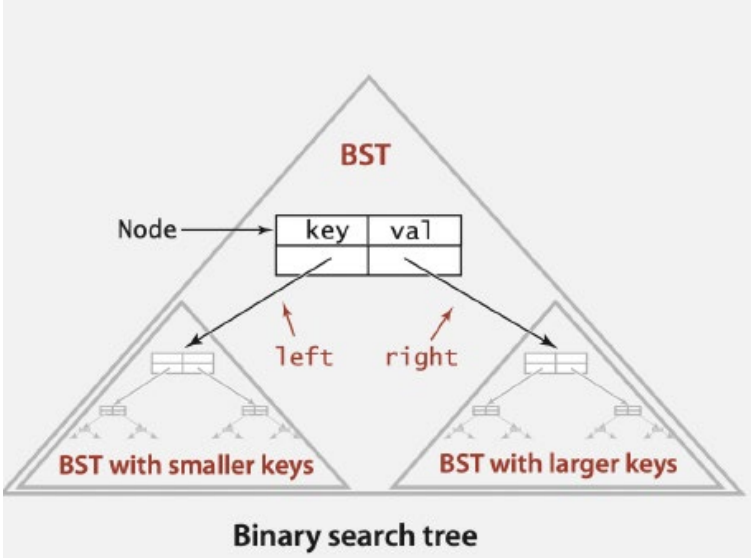
```
OutputStreamWriter dataOut =
        new OutputStreamWriter(new FileOutputStream("log.txt", true));
```

and then use the instance method **append** for dataOut.

2. In this task, you will use **binary search trees** to implement a so-called **Map** (also called a symbol table or dictionary). It is very similar to the **BST** implementation we created, but what is stored in a map are pairs consisting of a key and a value: we say that a value is associated with a key. You can think of this as a table, like an array where the indices do not need to be integers between 0 and the length. You should create the ***BSTMap.java*** file.
We first present the API and then provide a more detailed explanation:

```
public class BSTMap<T1 extends Comparable<T1>, T2>
---------------------------------------------------
// Create an empty table
public BSTMap()
// Add a key-value pair to the table.
// If there was already a pair with key, replace that with val.
public void put(T1 key, T2 val)
// The value associated with key in the table
public T2 get(T1 key)
// Is there a value associated with key in the table?
boolean contains(T1 key)
// Remove key (and value) from the table
public void delete(T key)
// Is the table empty?
public boolean isEmpty()
// Number of key-value pairs in the table
public int size()
// All keys in the table
Iterable<T1> keys()
```

The following Figure illustrates that the nodes must contain both a key and a value, and that only the key is used for ordering in the search:



Binary search tree

One can, for example, use String as T1 and Integer as T2. In that case, a Map can be used to count how many times each word appears in a text:

For each word that is read, the word is inserted into the table (the word is a key), either with value 1 associated (if it did not exist before) or with the previous value incremented by 1.

What is inserted using put is a word and an integer. If the word was not among the table's keys, a new node is created with the value 1 associated with the word. If the word already existed, the value is updated to the previous value plus 1.

a) (3 points): Implement the above API using binary search trees.

b) (1 point): Use the BSTMap class to write a program **WordCount** that reads a text from standard input and calculates how many times each word appears in the text. Create the **WordCount.java** file.
An example of running the program is

```
java WordCount
this is a sentence and this is not.
^D

a 1
and 1
is 2
not 1
sentence 1
this 2
```

Algorithms and data structures

DB4002
20 April 2026

HÖGSKOLAN I HALMSTAD

BONUS!!!

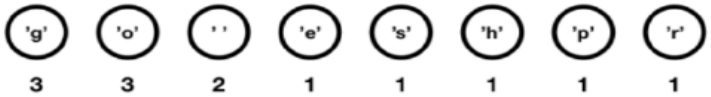
Assignments - part 2

3. (1 point): Focus of the task: binary trees, compression.
Binary trees can be used in many applications. One of these is an optimal lossless compression technique called **Huffman coding**. We will use this technique to compress text strings (i.e., create a new encoded representation that is shorter but contains all the information of the original string and can be transformed back into the original string).

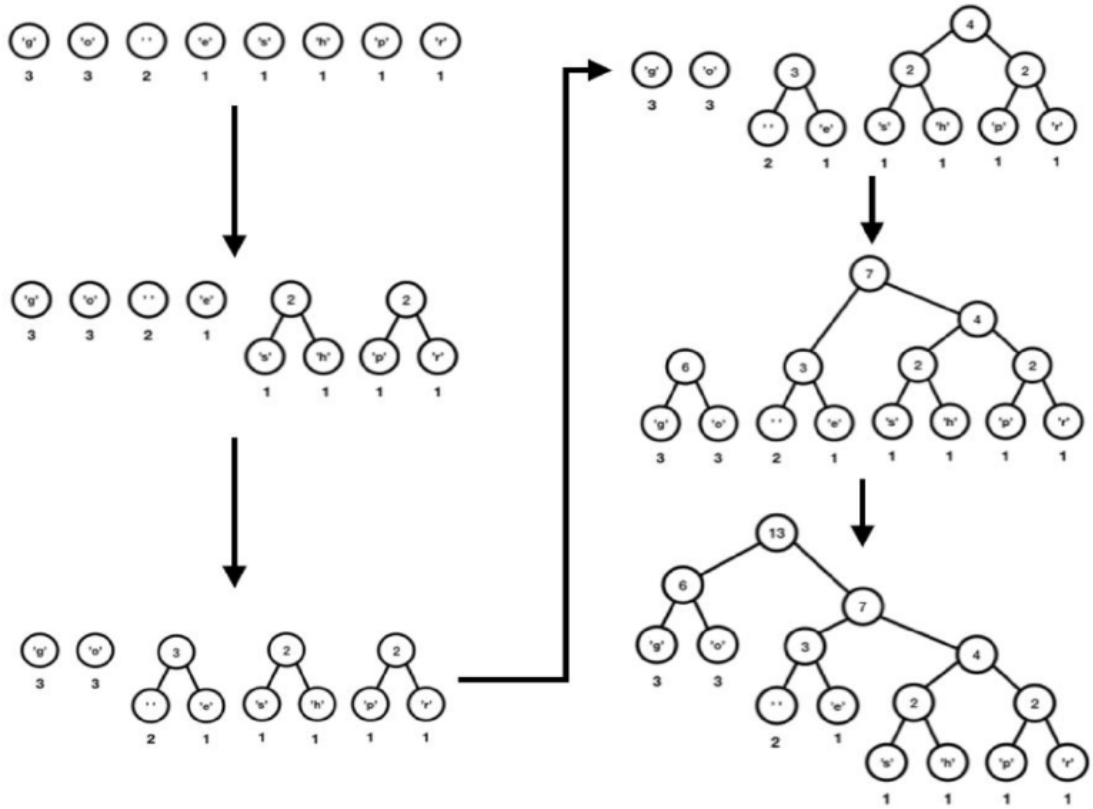
See a description of Huffman coding, for example here:
http://www2.cs.duke.edu/csed/poop/huff/info/
The process is also illustrated below with an example taken from the same source.

Step 1: If there are **n** unique characters in the string to be encoded, create **n** Huffman trees where each tree contains a node with the character and an integer weight equal to the number of occurrences of that character.

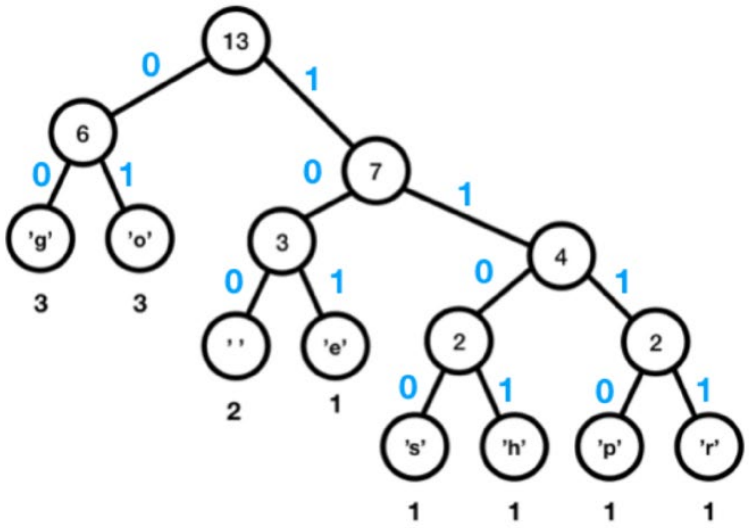
"go go gophers"



Step 2: Merge the Huffman trees until only one Huffman tree remains. The merging is done by selecting two trees whose combined weight is minimal (there may be several options; choose one of them). Multiple steps may be illustrated in the same Figure below.



Step 3: Create a binary representation for each character using the Huffman tree.



char	binary
'g'	00
'o'	01
'p'	1110
'h'	1101
'e'	101
'r'	1111
's'	1100
' '	100

Step 4: Encode the characters in the string using the binary representation.

"go go gophers" → 00011000001100000111101101101111111100

char	binary
'g'	00
'o'	01
'p'	1110
'h'	1101
'e'	101
'r'	1111
's'	1100
' '	100

13 bytes **go go gophers**

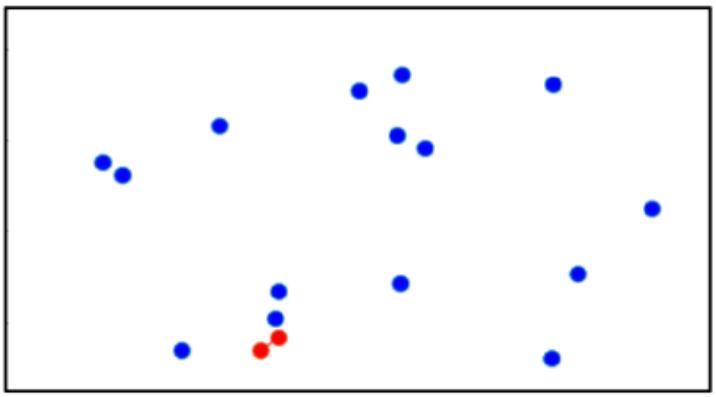
4 bytes och 5 bits 00011000|00110000|01111011|01011111|1100

We see that we can compress a string that originally used 13 bytes into something that uses 4 bytes and 5 bits.
If we want to represent the compressed data using characters, each character must be one byte. What can be done is to go back to **Step 1** and add a character that does not appear in the string and that represents the end of the encoding.

When encoding the string, you then append the binary representation of this character and pad with arbitrary bits so that the encoded message consists of a whole number of bytes. Then, during decoding, you stop decoding when this character has been decoded.

Java files are provided for the **Huffman tree** and node for use in the encoding (you should use the HuffmannTree.java file). Create the class **HuffmannCoding** in a file **HuffmanCoding.java**. The required functionality is to be able to encode a string using Huffman coding and then decode it back to the original string (create the main for a test, containing for example: String text = "this is a test for the lab HH";).

2. (1 point): In this task, you will implement a data type **PointSet** to represent a set of points (in the two-dimensional plane). See the figure below for an example of such a point set. In addition to being able to add and remove points from the set, we also want to quickly determine, for example, the minimum distance between two points (see the red points in the figure).



You are provided with the classes Point and PointPair in the attached files Point.java and PointPair.java. Create your **PointSet.java** file and the **PointSetTest.java** file for API testing.

In this implementation, you may use existing Java collections such as HashMap, TreeSet, or LinkedList. The **TreeSet** collection is implemented using a data structure called Red-Black binary trees, which is an implementation of binary search trees that maintains a balanced tree. Operations such as insertion and deletion have time complexity O(log n), where n is the number of values in the tree.

You may, for example, use a TreeSet to represent all pairs of points (PointPair), or use a HashMap to represent which points exist. You could also maintain a list of all points, etc. To implement closestPair(double distance), you may, for example, use the methods floor and ceil in TreeSet.

The following API describes the data type you are to implement. Note that the description also specifies the expected computational complexity for each method when the set contains **n** points.

```
public class PointSet implements Iterable<Point>
-----------------------------------------------------
// Create the empty set.
// O(1).
```

Algorithms and data structures

DB4002
20 April 2026

HÖGSKOLAN I HALMSTAD

```
public PointSet()
// Make the set empty.
// O(1).
public void clear()
// Is the set empty?
// O(1).
public boolean isEmpty()

// Add the point p.
// Return true if p already exists otherwise false.
// O(n log(n)). (If you find an O(1) implementation, it is also O(n log(n))!)
public boolean addPoint(Point p)

// Is p in the set?
// O(n log(n)). (Om ni hittar en implementation O(1) □ar den ocks_a O(n log(n))!)
public boolean contains(Point p)

/// Remove p from the set.
// O(n log(n)). (If you find an O(1) implementation, it is also O(n log(n))!)
public boolean remove(Point p)
// Number of points in the set.
// O(1).
public int size()

// The point pair that has the shortest distance between the two points.
// O(1).
public PointPair minDistance()

// The point pair that has the longest distance between the two points.
// O(1).
public PointPair maxDistance()

// The point pair whose distance between the two points is the smallest distance.
// O(n log(n)).
public PointPair closestPair(double distance)

// Iterate over the points in the set.
// O(n).
public Iterator<Point> iterator()

// Iterate over the sorted point pairs that contain the point p.
// The point pair (PointPair) that has the shortest distance comes first,
// the one with the closest distance comes last, and so on. // O(n).
public Iterable<PointPair> pointDistIterable (Point p)

// Iterate over all the sorted PointPairs.
// The PointPair with the shortest distance comes first,
// the one with the next shortest distance comes last, etc. // O(n*n)
public Iterable<PointPair> allDistIterable()
```